



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIO DE PRÁCTICA N° 06

NOMBRE COMPLETO: EMILIANO GARCIA OLVERA

N° de Cuenta: 314256009

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 14/MARZO/2026

CALIFICACIÓN: _____

Ejercicio 1: Crear un dado de 8 caras y texturizarlo por medio de código.

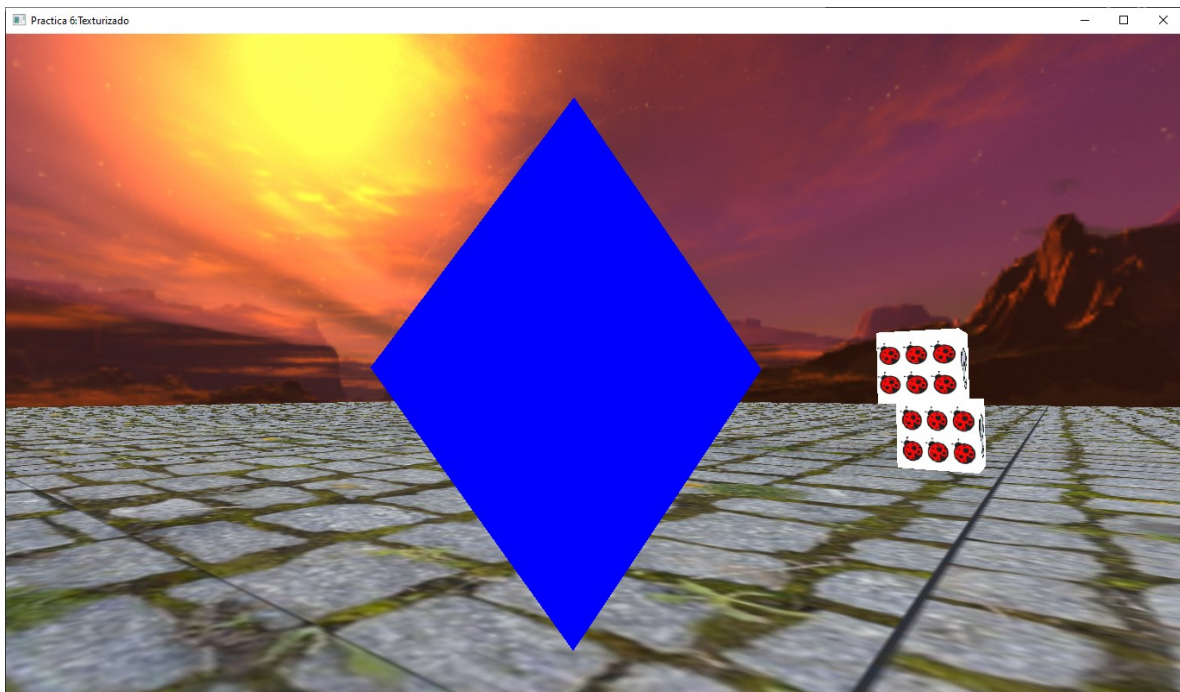
Creando el dado de 8 caras

Para el primer ejercicio, comenzamos simple. Solamente creando la forma general del dado de 8 caras a partir de los siguientes vertices e indices definidos.

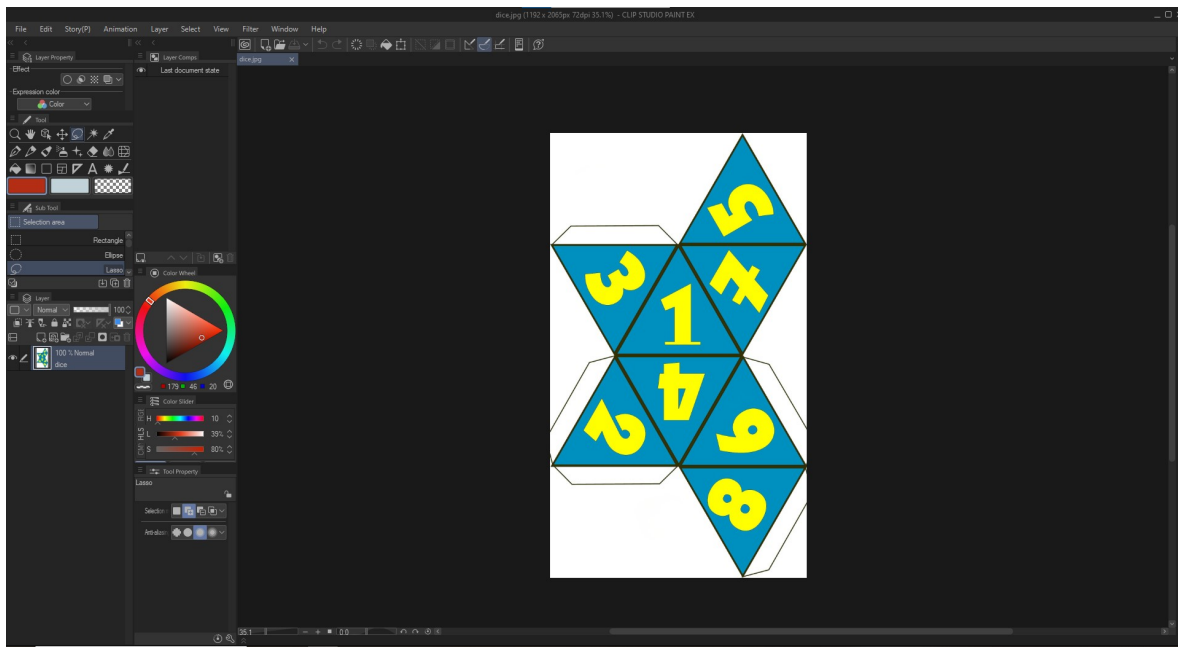
```
void CrearPiramideCuadrangular()
{
    std::vector<unsigned int> indices = {
        0,3,4,
        3,2,4,
        2,1,4,
        1,0,4,
        0,3,5,
        3,2,5,
        2,1,5,
        1,0,5,
    };

    std::vector<GLfloat> vertices = {
        0.5f,-0.0f,0.5f,
        0.5f,-0.0f,-0.5f,
        -0.5f,-0.0f,-0.5f,
        -0.5f,-0.0f,0.5f,
        0.0f, 1.0f,0.0f,
        0.0f,-1.0f,0.0f,
    };

    Mesh* piramide = new Mesh();
    piramide->CreateMeshGeometry(vertices, indices,
vertices.size(), indices.size());
    meshList.push_back(piramide);
}
```



Seguido de esto comenzamos con la texturización. Conseguimos una plantilla de un dado de 8 caras en internet y lo adaptamos de manera que pueda ser mas facil de manipular, donde los vertices caen en cuartos o tercios de la imagen.



Esto se hizo en Clip Studio Paint. Un software de manipulacion de imagenes que es una alternativa a photoshop y gimp, y que utilice por ser con el que estoy mas familiarizado.

Y exportamos para obtener una imagen asi.



Ahora modificaremos el código anterior de la siguiente manera. Crearemos tres vértices por cada cara y procederemos a colocar las coordenadas UV. Generalmente caerán en tercios o cuartos, sin embargo cada vértice tuvo que ser posicionado precisamente para terminar con un buen resultado.

```
void CrearDado2()
{
    std::vector<unsigned int> indices = {
        0, 1, 2,
        3, 4, 5,
        6, 7, 8,
        9, 10, 11,
        12, 13, 14,
        15, 16, 17,
        18, 19, 20,
        21, 22, 23,
    };

    // std::vector<GLfloat> vertices = {
        vec1  vec2  vec3      u      v
        -0.5f, 0.0f, 0.5f,    0.3f, 0.51f, // 1
        0.5f, 0.0f, 0.5f,    0.7f, 0.51f,
        0.0f, 1.0f, 0.0f,    0.5f, 0.72f,

        0.5f, 0.0f, 0.5f,    0.25f, 0.49f, // 2
        0.5f, 0.0f, -0.5f,   0.04f, 0.26f,
        0.0f, 1.0f, 0.0f,    0.47f, 0.26f,

        0.5f, 0.0f, -0.5f,   0.06f, 0.74f, // 3
        -0.5f, 0.0f, -0.5f,   0.25f, 0.51f,
        0.0f, 1.0f, 0.0f,    0.48f, 0.74f,

        -0.5f, 0.0f, -0.5f,   0.3f, 0.49f, // 4
        -0.5f, 0.0f, 0.5f,    0.7f, 0.49f,
        0.0f, 1.0f, 0.0f,    0.5f, 0.29f,

        -0.5f, 0.0f, 0.5f,    0.94f, 0.76f, // 5
        0.5f, 0.0f, 0.5f,    0.75f, 0.98f,
        0.0f, -1.0f, 0.0f,    0.52f, 0.76f,

        0.5f, 0.0f, 0.5f,    0.95f, 0.26f, // 6
        0.5f, 0.0f, -0.5f,   0.75f, 0.49f,
        0.0f, -1.0f, 0.0f,    0.52f, 0.26f,

        0.5f, 0.0f, -0.5f,   0.75f, 0.51f, // 7
        -0.5f, 0.0f, -0.5f,   0.97f, 0.74f,
        0.0f, -1.0f, 0.0f,    0.52f, 0.74f,

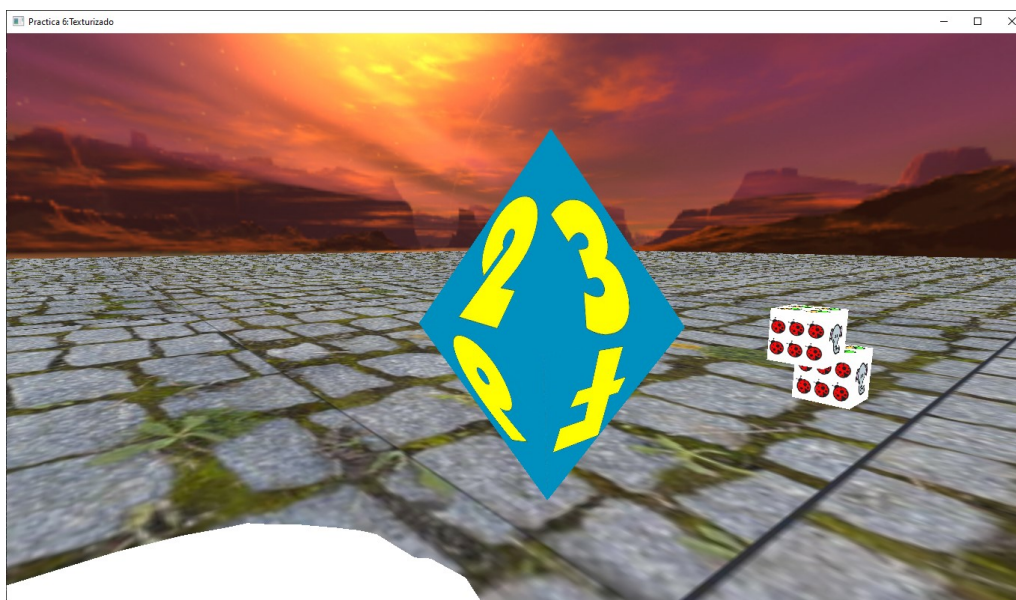
        -0.5f, 0.0f, -0.5f,   0.75f, 0.03f, // 8
        -0.5f, 0.0f, 0.5f,    0.97f, 0.23f,
        0.0f, -1.0f, 0.0f,    0.52f, 0.24f,
    };

    Mesh* piramide = new Mesh();
    piramide->CreateMeshGeometry(vertices, indices, vertices.size(),
indices.size());
    meshList.push_back(piramide);
}
```

Finalmente declaramos la textura, la cargamos y la utilizamos como textura en la figura que creamos anteriormente.

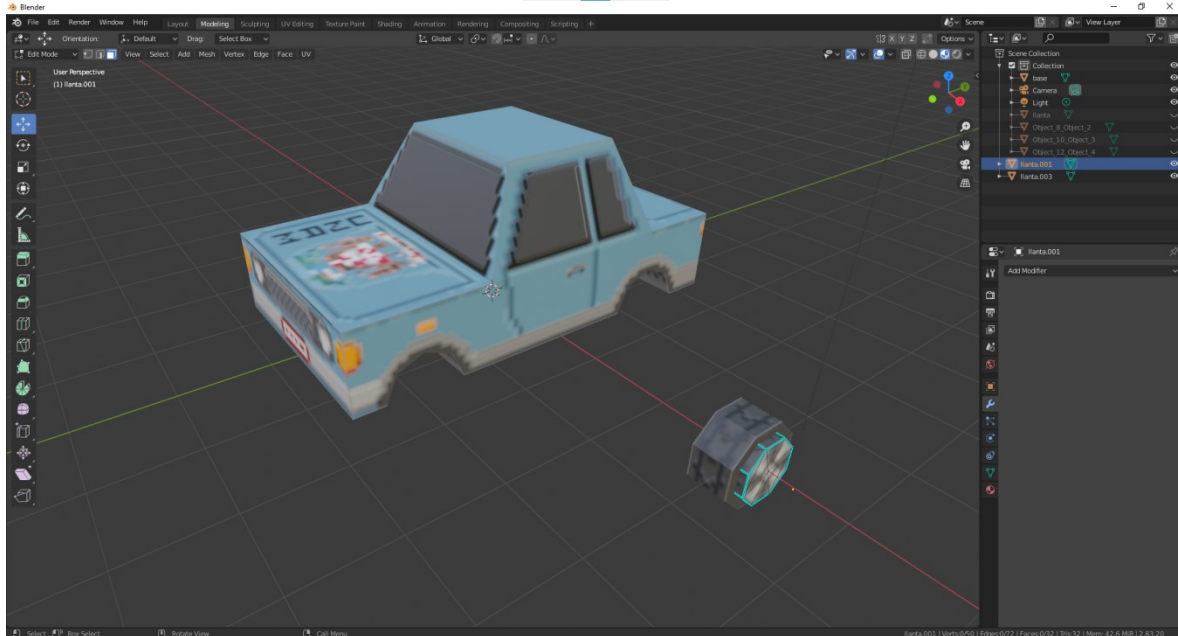
```
Texture diceTexture;  
  
diceTexture = Texture("Textures/dice.jpg");  
diceTexture.LoadTextureA();  
  
//DADo  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
dadoTexture.UseTexture();  
meshList[4]->RenderMesh();
```

Y listo.



Ejercicio 2: Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin)

Primero, importamos el modelo de nuestro carro en blender y nos aseguramos de que tengamos todas las piezas para poder trabajar con este. Separamos las llantas y exportamos el caucho y el rin como nos pide el ejercicio.



Seguido de esto, el procedimiento estandar. Declaramos los modelos, los cargamos con los modelos recién exportados y los posicionamos en el mundo.

```
Model carro_model;  
Model caucho_model;  
Model rin_model;
```

```
carro_model = Model();  
carro_model.LoadModel("Models/carro.obj");  
caucho_model = Model();  
caucho_model.LoadModel("Models/llanta-caucho.obj");  
rin_model = Model();  
rin_model.LoadModel("Models/llanta-rin.obj");
```

```
//Ejercicio 2: Importar el modelo de su coche con sus 4 llantas acomodadas  
//y tener texturizadas las 4 llantas (diferenciar caucho y rin) y  
//texturizar el logo de la Facultad de ingeniería en el cofre de su propio  
modelo de coche
```

```
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(8.0f, -1.0f, 2.0f));  
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));  
modelaux = model;  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
carro_model.RenderModel();  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);  
model = modelaux;  
model = glm::translate(model, glm::vec3(1.2f, 0.0f, 1.85f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
caucho_model.RenderModel();
```

```

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(1.2f, 0.0f, 1.85f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
rin_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(-1.2f, 0.0f, 1.85f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0, 1.0, 0.0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
caucho_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(-1.2f, 0.0f, 1.85f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0, 1.0, 0.0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
rin_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(1.2f, 0.0f, -1.39f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
caucho_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(1.2f, 0.0f, -1.39f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
rin_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(-1.2f, 0.0f, -1.39f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0, 1.0, 0.0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
caucho_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
model = modelaux;
model = glm::translate(model, glm::vec3(-1.2f, 0.0f, -1.39f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0, 1.0, 0.0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
rin_model.RenderModel();
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);

```

Notaremos glTexParameter ya que no habiamos trabajado con esta. Y es que nuestro modelo es perfecto para observar su funcionamiento. Estas basicamente setean OpenGL para que nuestro modelo utilice el algoritmo de Nearest Neighbors, el cual produce ese efecto pixel art donde no interpolamos nuestros valores. De no usarlo obtendriamos un resultado como se ve en blender en el que con este estilo, la textura del coche se veria borrosa.



Al final no nos olvidamos como en la practica anterior de modificar nuestro coche para que el cofre y las llantas puedan moverse (obteniendo los eventos de las teclas y aumentando a las variables de window.h).

Y finalmente agregaremos unos ultimos toques tanto al cofre y parrilla, como el parabrisas para que nuestro coche tenga ojos tipo cars



Y finalizamos con esta practica.



Conclusión

Durante esta práctica se cumplieron los tres objetivos planteados. En primer lugar, se utilizó Clip Studio Paint como herramienta de edición de imágenes para adaptar una plantilla de dado de 8 caras, ajustando las posiciones de los números de modo que las coordenadas UV cayeran en fracciones manejables (tercios y cuartos), lo cual demuestra la importancia de preparar adecuadamente las texturas antes de aplicarlas a un modelo 3D. En segundo lugar, se empleó la librería de carga de imágenes para cargar archivos JPG como texturas en OpenGL, utilizando funciones como `LoadTextureA()` y `UseTexture()` para vincularlas al pipeline de renderizado. Finalmente, se comprendió cómo asociar información de texturizado a la geometría mediante coordenadas UV, asignando a cada vértice del octaedro sus valores correspondientes para mapear correctamente cada cara del dado con su número. Esta misma lógica se extendió al ejercicio del coche, donde se diferenciaron texturas para el caucho y el rin de las llantas, y se exploró el uso de `glTexParameterI` con `GL_NEAREST` para lograr un estilo pixel art sin interpolación, reforzando así la comprensión del proceso completo de texturizado en OpenGL.

Referencias:

- Apuntes de la clase de teoría y laboratorio de “Computación Gráfica e Interacción Humano Computadora”
- De Vries, J. (n.d.). Coordinate systems. LearnOpenGL. learnopengl.com